Update **confidential** designator here

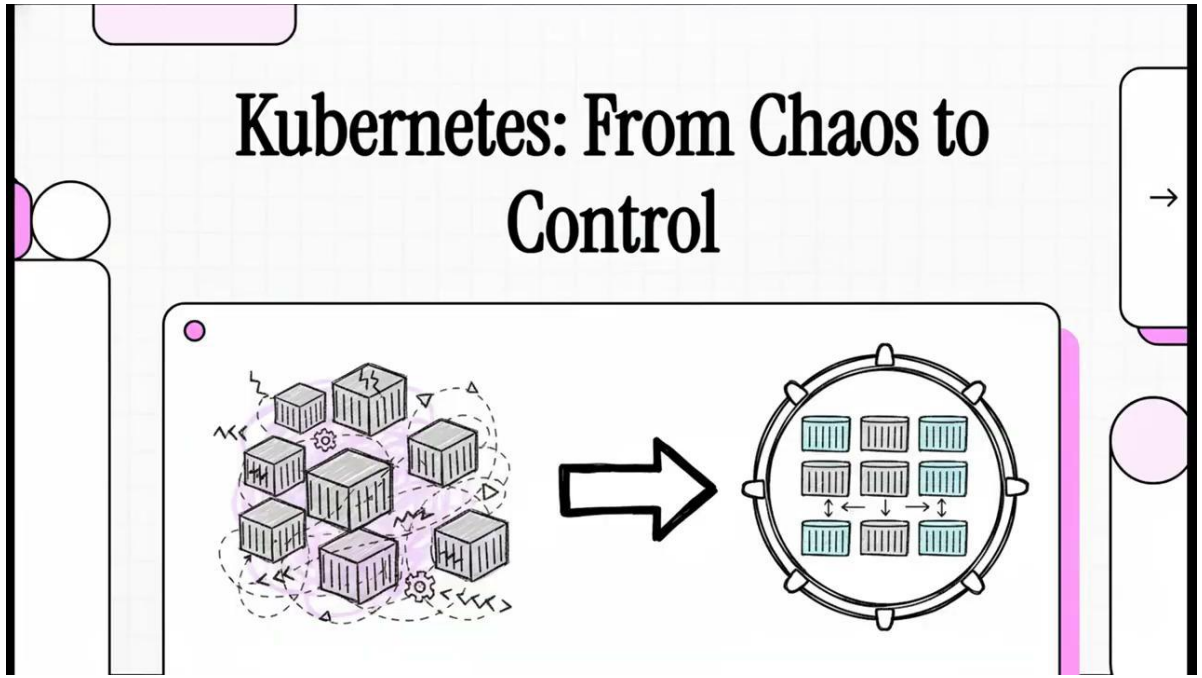
OpenShift Architecture – an Overview

Technical Overview for Architects

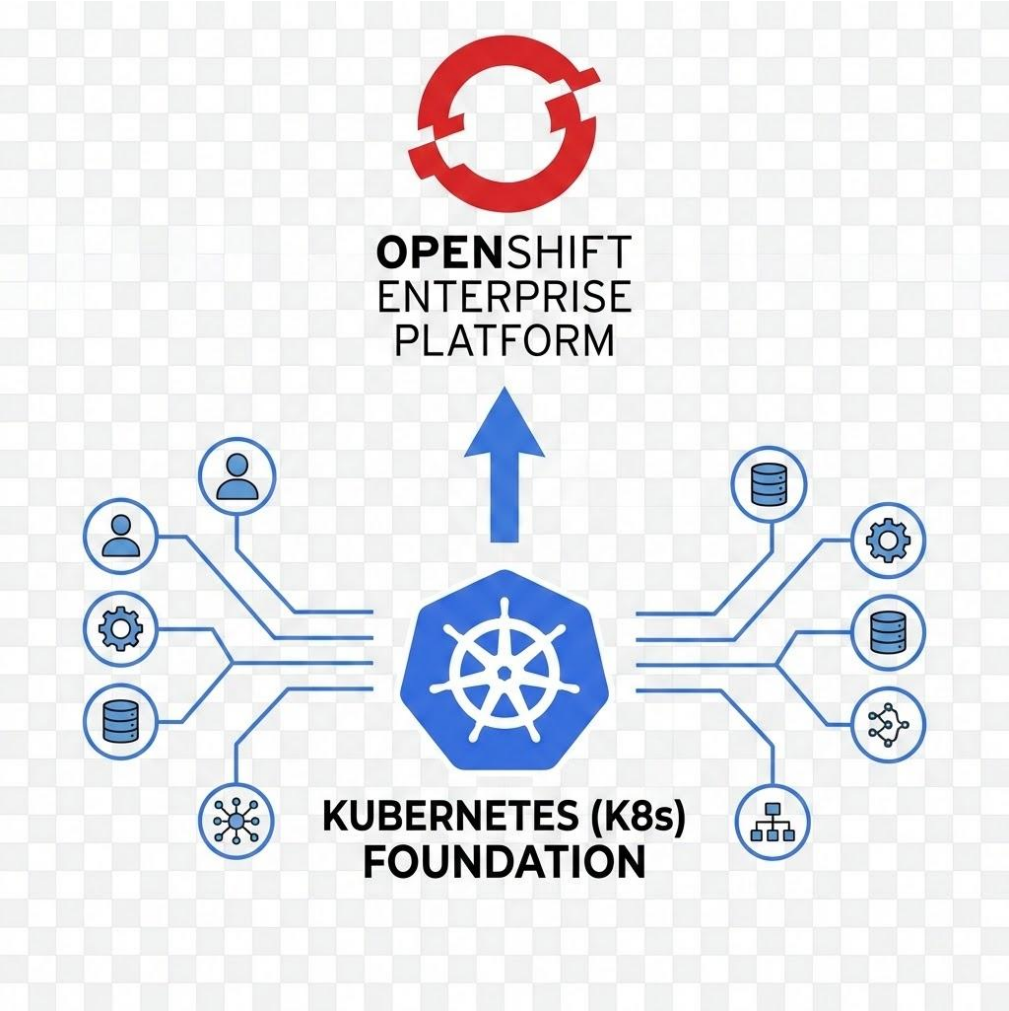
Red Hat

Agenda

- OpenShift vs. Kubernetes
- Overall architecture
- Control Plane components
- Worker Nodes
- etcd – cluster state store
- Kubelet – node agent
- Scheduler – pod placement
- Orchestration process



OpenShift and Kubernetes



Red Hat

Update **confidential** designator here

Similarities and Differences

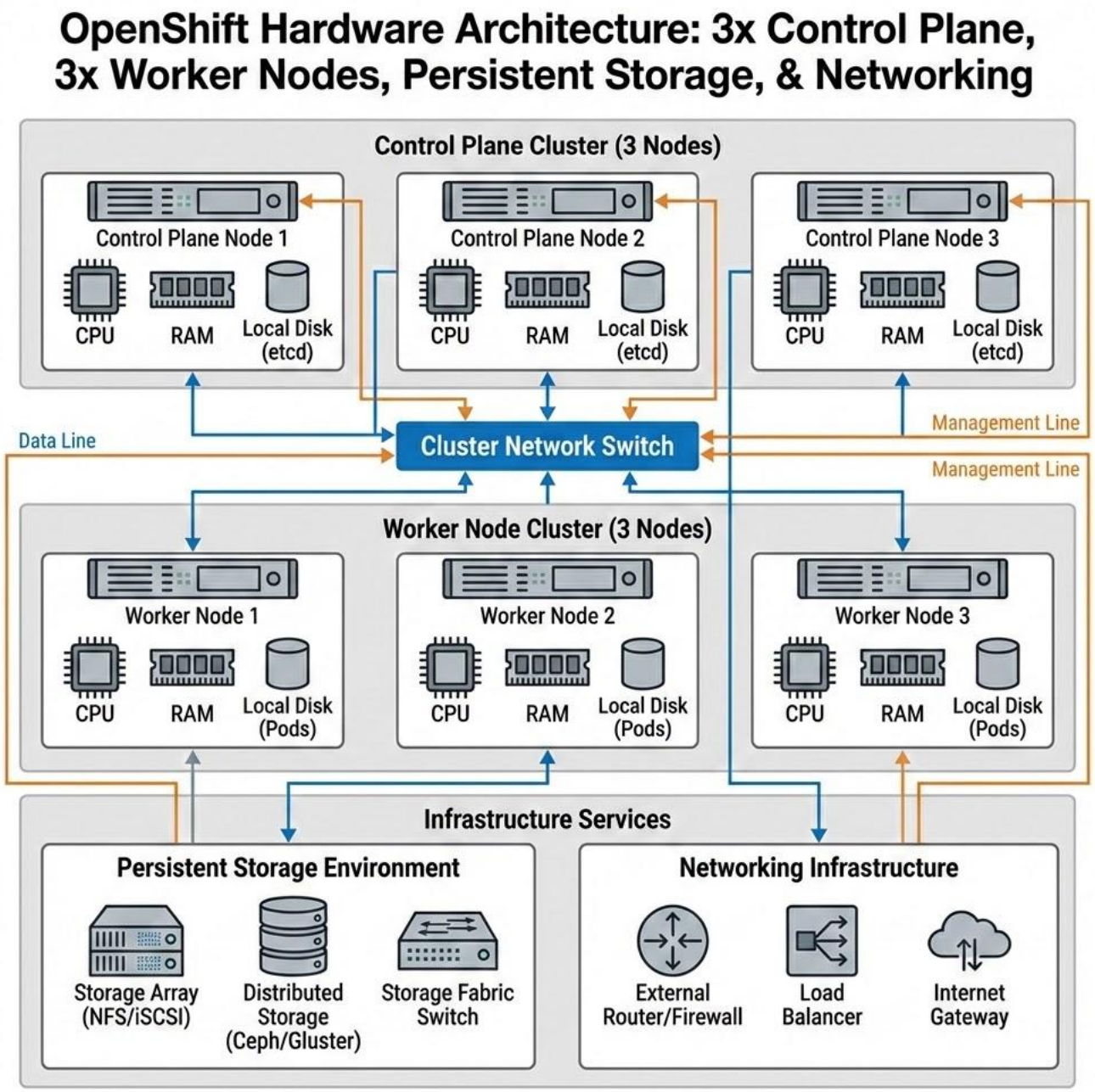
Same foundation, different value

- Same core architecture (Control Plane, Workers)
- Identical components: etcd, Scheduler, Kubelet
- Kubernetes API and resources (Pods, Deployments)
- OpenShift is certified Kubernetes distribution
- Same container runtime concepts (CRI-O)
- OpenShift: Enterprise support and lifecycle
- Built-in registry, CI/CD (Pipelines)
- Operator Lifecycle Manager (OLM)
- Enhanced RBAC and security defaults
- Web console and oc CLI (kubectl compatible)

Red Hat

OpenShift extends Kubernetes with enterprise features.

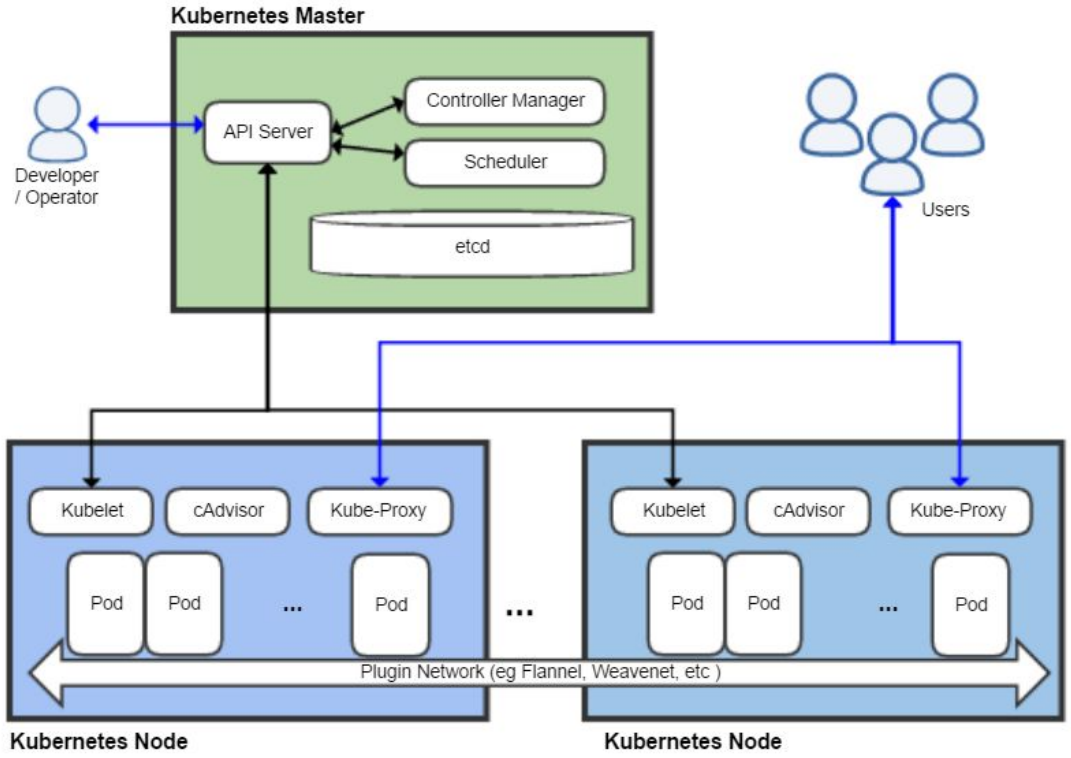
Architecture Overview



Overall Architecture

Control Plane and Worker Nodes

- Control Plane: Cluster brain and API
- Worker Nodes: Run your workloads
- Shared etcd for distributed state
- Networking connects all components

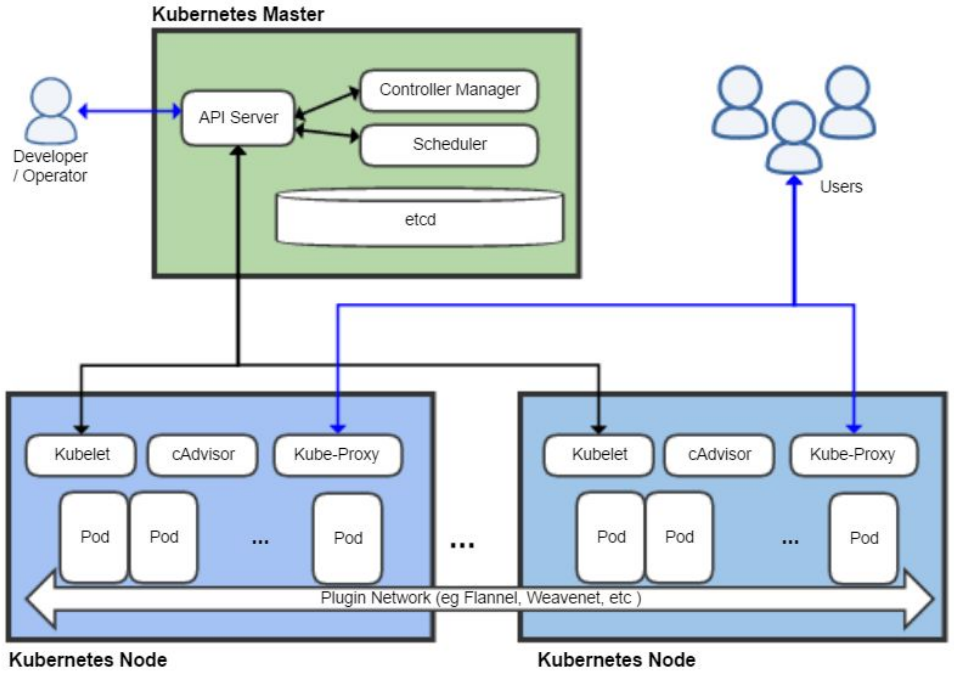


Source: kubernetes.io/docs/concepts/architecture, Khtan66/Wikimedia (CC BY-SA 4.0)

Control Plane

Core components

- API Server: Single entry point for all requests
- etcd: Distributed key-value store
- Scheduler: Assigns pods to nodes
- Controller Manager: Maintains desired state



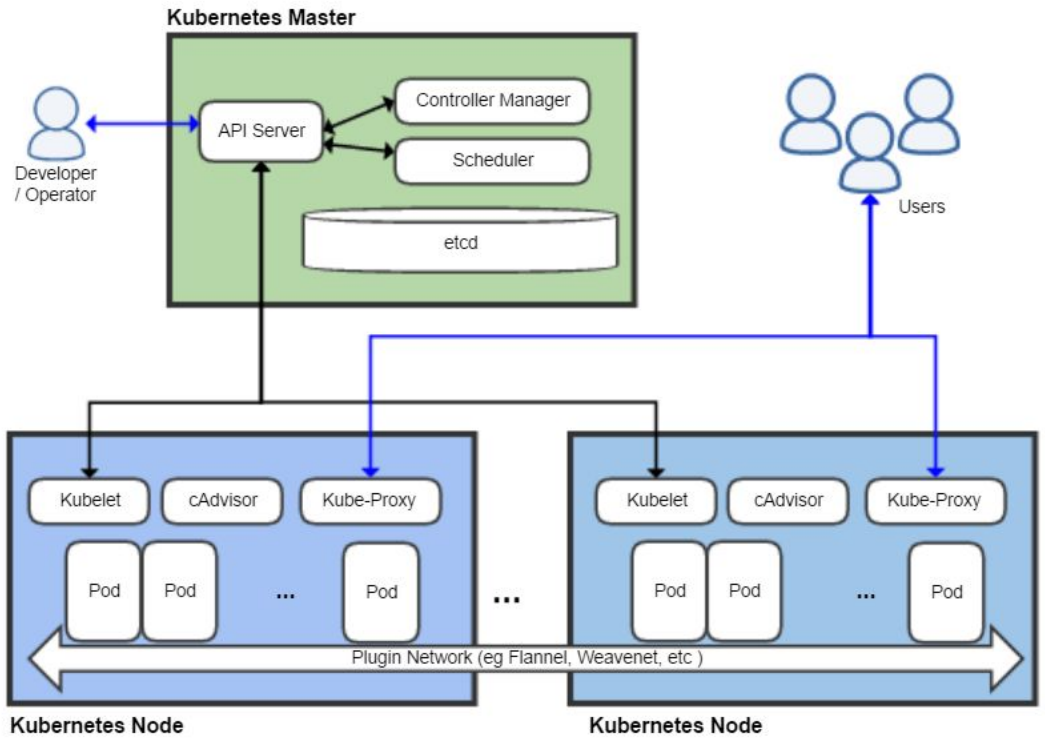
Source: kubernetes.io/docs/concepts/architecture

Update **confidential** designator here

Worker Nodes

Where containers run

- Run the actual pods and containers
- Kubelet reports status to Control Plane
- Container runtime (CRI-O) runs containers
- kube-proxy handles network rules

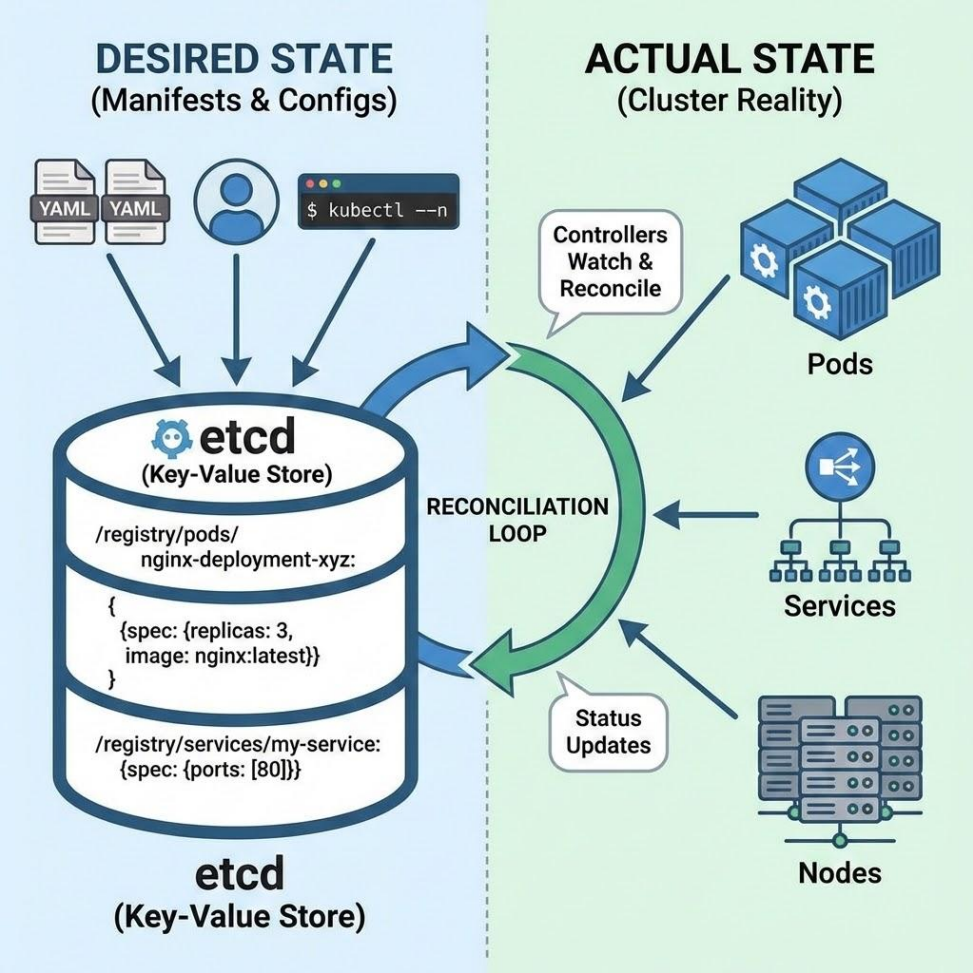


Source: kubernetes.io/docs/concepts/architecture

etcd

Distributed key-value database

- Stores all cluster configuration
- Consensus via Raft for consistency
- Source of truth for cluster state
- Highly available in production setups



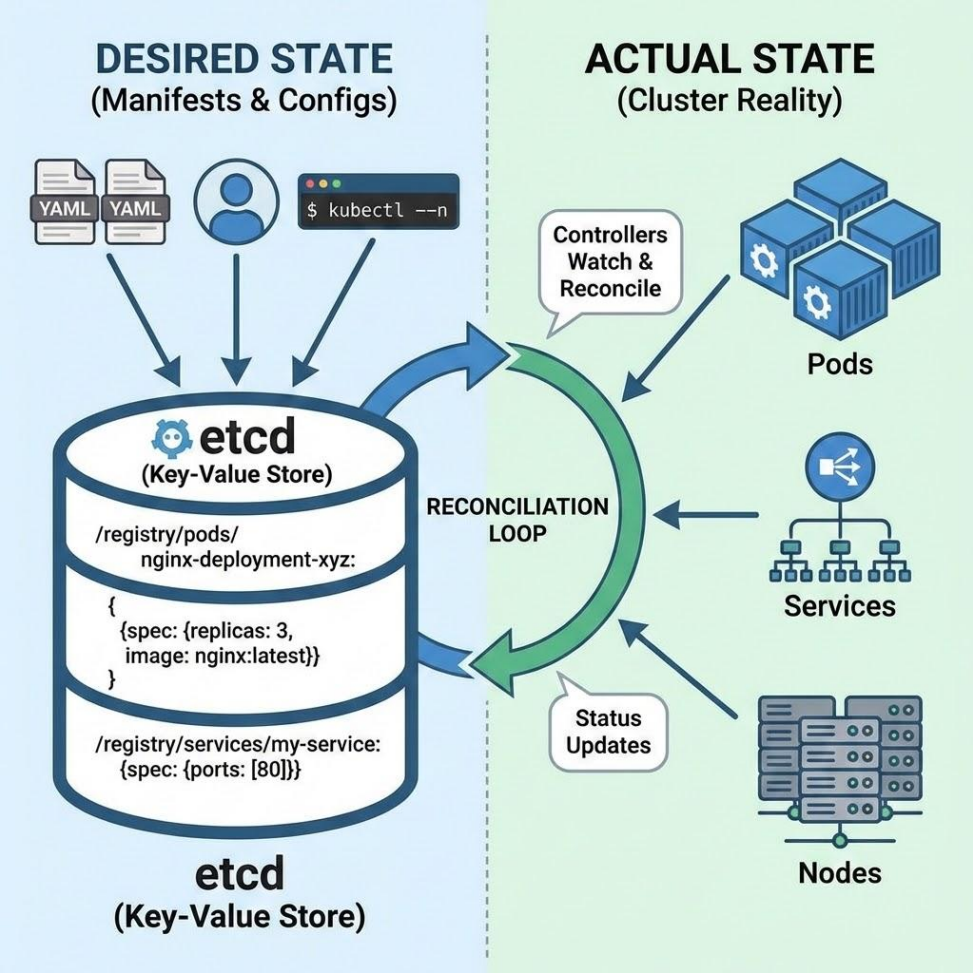
Source: etcd.io, CNCF Artwork

Red Hat

Kubelet

Runs on every worker node

- Receives pod specs from API Server
- Manages pod lifecycle on the node
- Reports node and pod status
- Ensures containers stay healthy

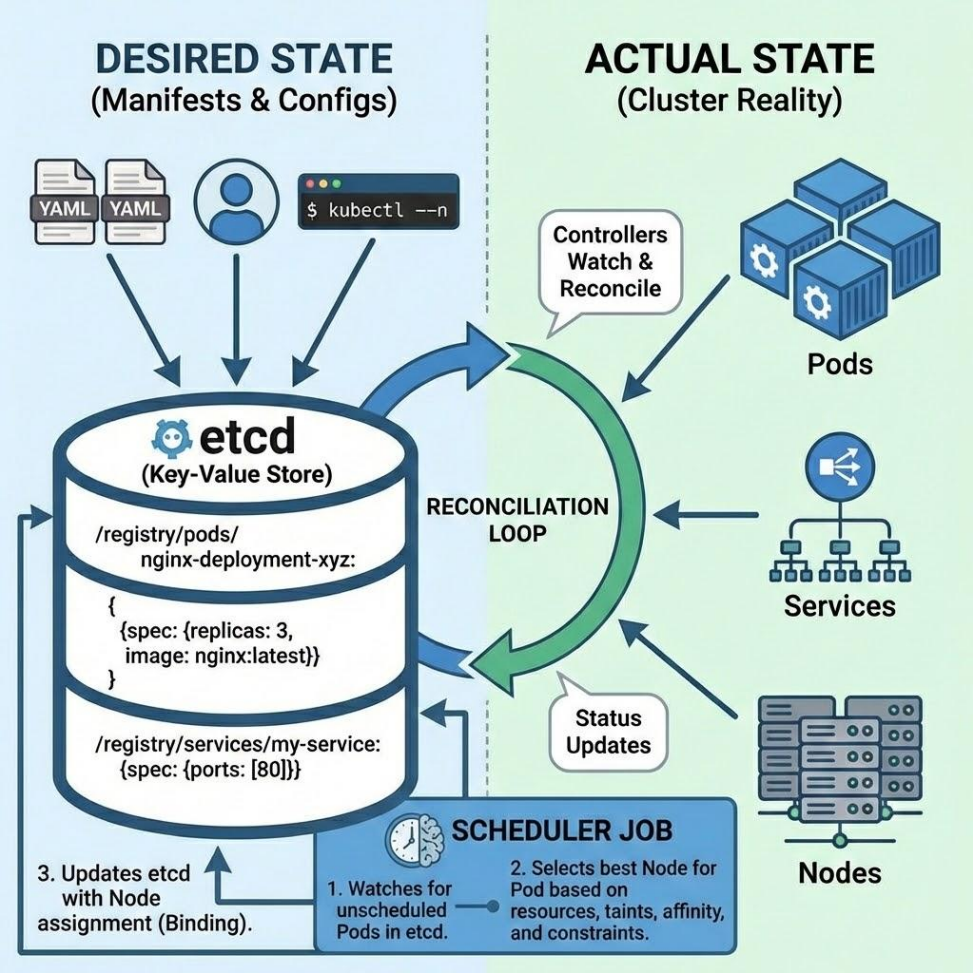


Source: kubernetes/community icons

Scheduler

Decides where pods run

- Watches for unscheduled pods
- Evaluates node resources and constraints
- Assigns pods to suitable nodes
- Respects affinity and tolerations



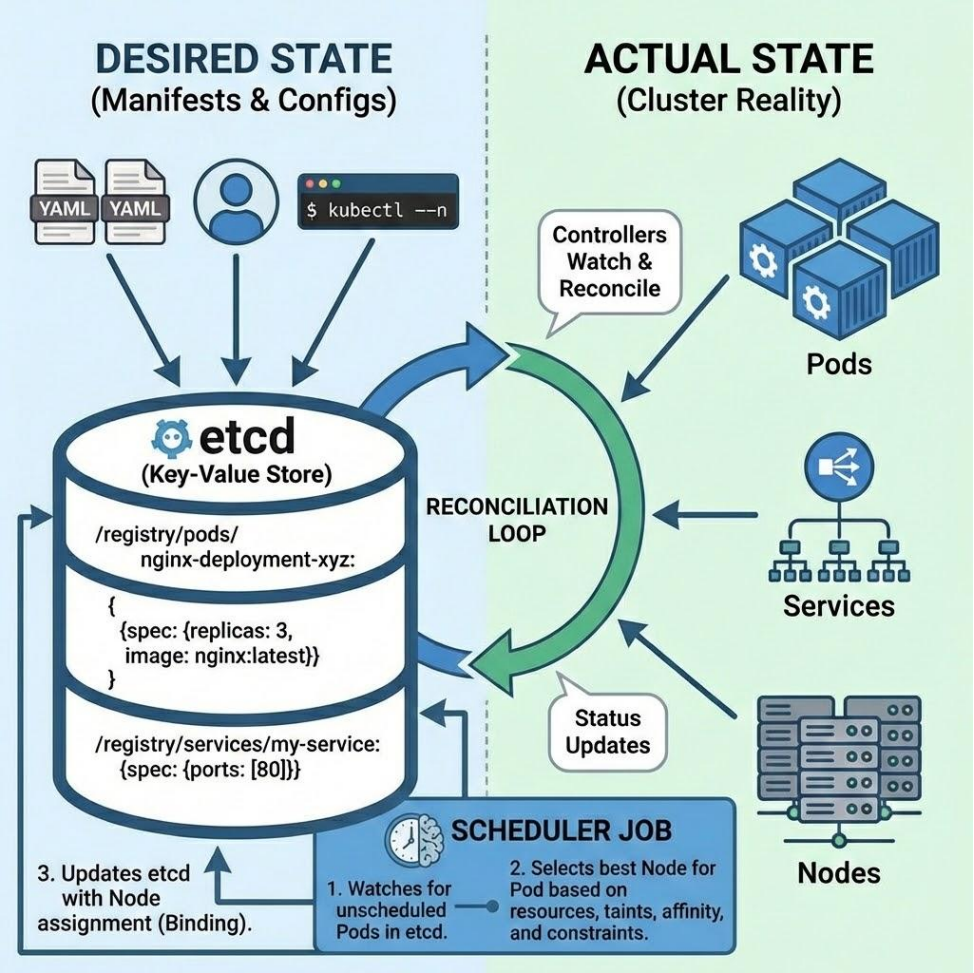
Source: kubernetes/community icons

Orchestration Process

Orchestration: Step 1

User initiates the request

- ① User applies YAML via API or oc/kubectl
- Request enters the cluster through API Server



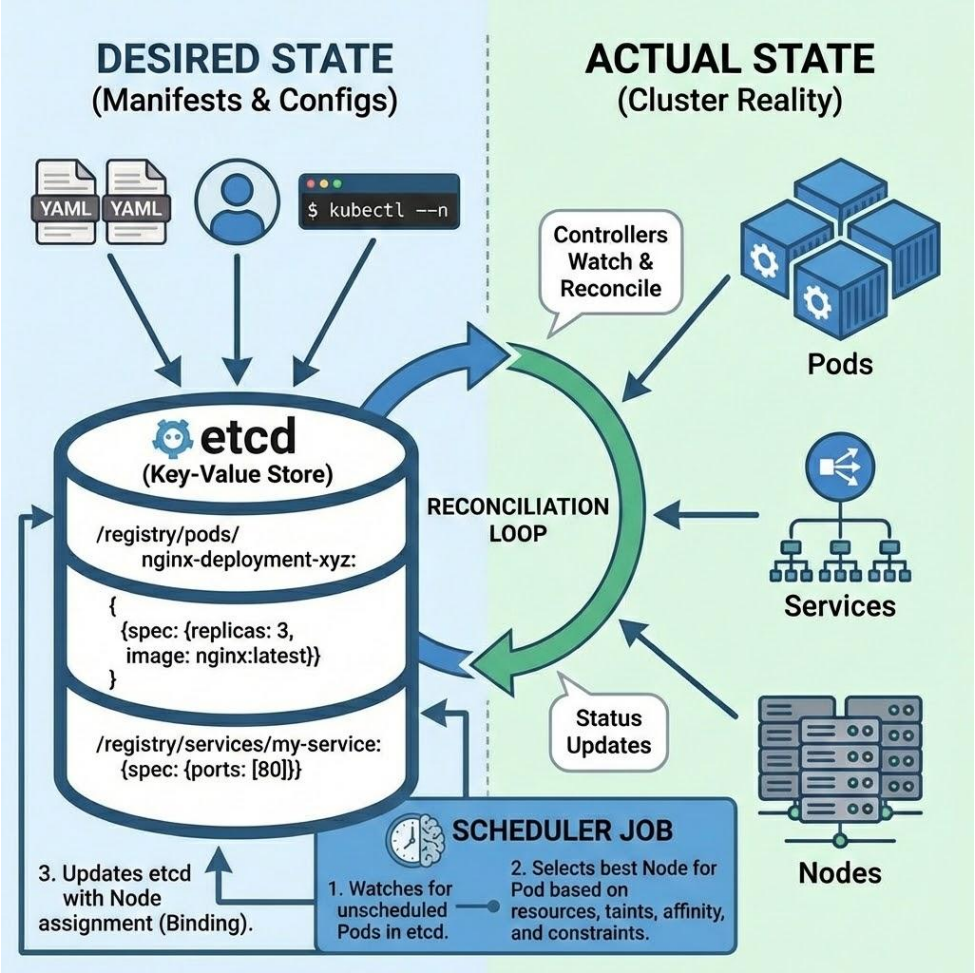
Source: kubernetes.io/docs/concepts/architecture

Update **confidential** designator here

Orchestration: Step 2

API Server persists state

- ① User applies YAML via API or oc/kubectl ✓
- ② API Server validates and writes to etcd
- Cluster state is stored in etcd



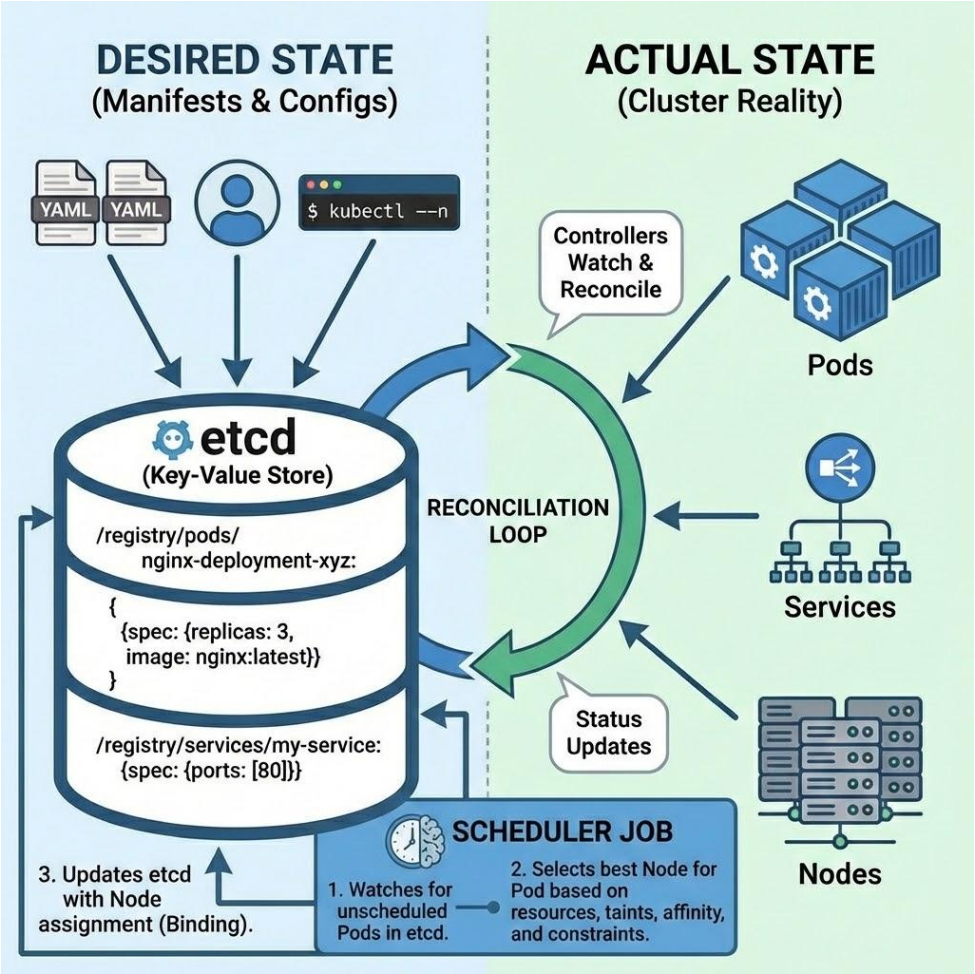
Source: kubernetes.io/docs/concepts/architecture

Update **confidential** designator here

Orchestration: Step 3

Scheduler assigns the pod

- ① User applies YAML via API or oc/kubectl ✓
- ② API Server validates and writes to etcd ✓
- ③ Scheduler picks a node for each pod
- Node selection based on resources and constraints



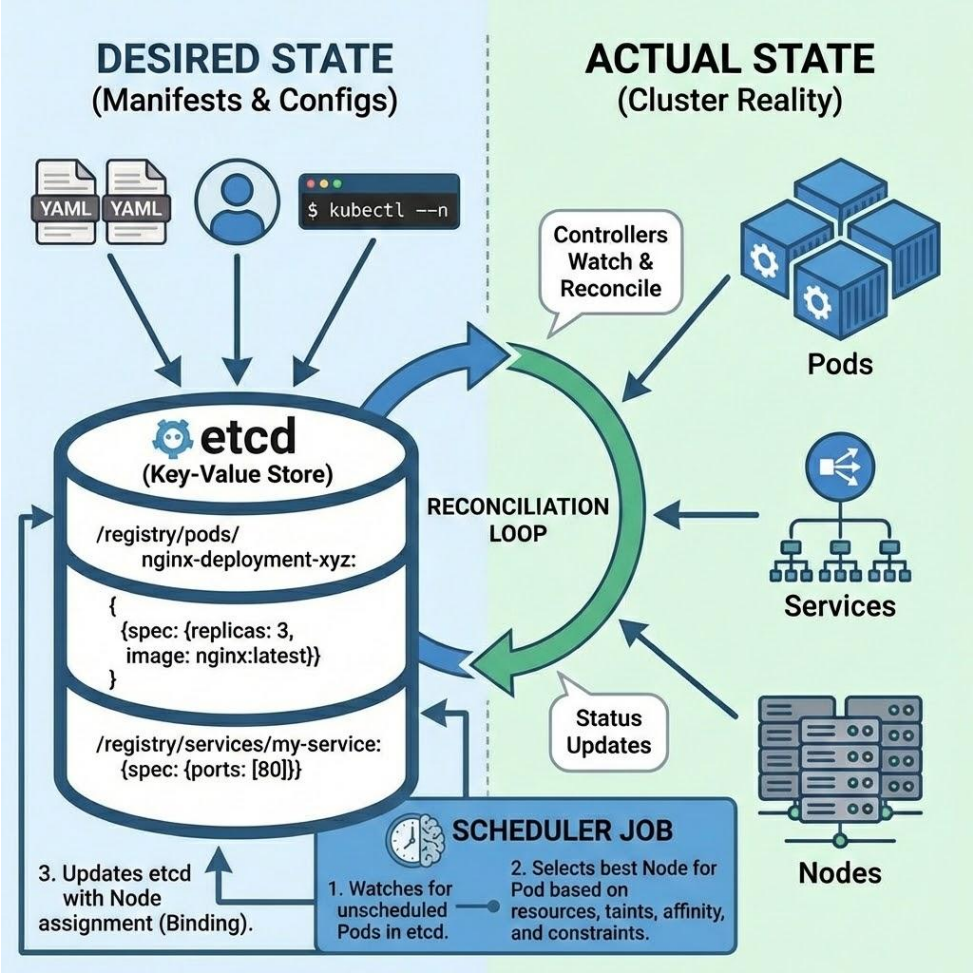
Source: kubernetes.io/docs/concepts/architecture

Update **confidential** designator here

Orchestration: Step 4

Kubelet runs the containers

- ① User applies YAML via API or oc/kubectl ✓
- ② API Server validates and writes to etcd ✓
- ③ Scheduler picks a node for each pod ✓
- ④ Kubelet pulls image and starts containers
- → Pod is running on the assigned node



Source: kubernetes.io/docs/concepts/architecture

Thank You

Questions? Reach out to your Red Hat team.

linkedin.com/company/red-hat

youtube.com/user/RedHatVideos

facebook.com/redhatinc

twitter.com/RedHat

Red Hat